IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

Training, Diagnosing and Adapting a Low-Cost No-Limit Hold'em Agent on a Single Laptop

SAKSHAM DUBEY[1]
[1]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu 632014, India (e-mail: saksham.dubey2024@vitstudent.ac.in)
Corresponding author: Saksham Dubey (e-mail: saksham.dubey2024@vitstudent.ac.in).

[Funding: none, or name the source.]

ABSTRACT Superhuman poker agents compute approximate equilibria on large clusters and do not adapt to opponents. We study how far a single consumer laptop can go, and how to find where such an agent is weak. We train heads-up No-Limit Texas Hold'em strategies with parallel, linearly weighted Monte Carlo counterfactual regret minimization at about two hundred thousand iterations per second on twelve cores, after checking the solvers against published values on Kuhn and Leduc poker. We measure strength with local best response, an exploiting opponent that knows the strategy it faces, restricted to one betting round at a time to locate leaks. The river is the weak point: its leak grows from 276 to 736 milli-big-blinds per hand between two and sixty million iterations as the strategy overfits its card abstraction, and richer river buckets do not help. Solving the river during play, hand by hand, removes the leak and cuts total exploitability after the flop by seventy percent. We also show that a common way of seeding parallel random number generators silently correlates the samples of evaluation threads and understates error margins. Finally, an adaptation layer that leaves the trained strategy only when public evidence, compared with the strategy's own behavior in each situation, says it pays, multiplies the win rate by two to seven against flawed opponents, including one that switches style to punish adaptation, and stays close to the trained strategy against sound ones. The game engine, solvers and evaluation tools are released as open source.

INDEX TERMS Counterfactual regret minimization, exploitability, game theory, imperfect-information games, local best response, Monte Carlo methods, opponent modeling, poker, Texas Hold'em

I. INTRODUCTION

POKER has been a benchmark for decision making under uncertainty for decades. Two players see different information, chance deals the cards, and success depends on hiding one's own strength as much as on judging the opponent's. Agents such as Libratus and Pluribus now beat top professionals at No-Limit Texas Hold'em (NLHE) by computing an approximate Nash equilibrium [1], [2]. They are expensive to build and do not change their play for the person in front of them: against a player who folds too often, an equilibrium strategy collects only what that player's mistakes happen to give away.

This article asks three practical questions about agents built on modest hardware. How strong a strategy can one consumer laptop train? How can a developer find *where* such a strategy is weak, rather than only how weak it is overall? And can it be adapted to individual opponents without becoming easy to exploit itself?

We answer them with an open system that spans the whole path from game rules to a playable agent: a rules engine and network game server written in Java, a solver written in Rust, and an evaluation harness. Our contributions are:

1) A laptop-scale training pipeline for heads-up NLHE: suit-isomorphic card abstraction with distribution-aware buckets, and linearly weighted external-sampling Monte Carlo counterfactual regret minimization (MCCFR) on all cores with a lock-free shared table. It runs at about 2×10^5 iterations per second on twelve cores, and every component is checked against published values.
2) A street-level diagnosis of exploitability. Local best response (LBR) [3] restricted to one betting round shows that the flop of our strategies is close to sound while the river leaks, and that the river leak *grows* with training: the strategy overfits its card abstraction [4], [5] even as it keeps beating less-trained versions of itself. Richer river buckets that describe a hand by its strength against clusters of opponent hands do not reduce the leak.
3) River re-solving: solving the river subgame during play for every hand of both ranges removes the river leak

(from 736 to 7 mbb/hand) and cuts exploitability after the flop by 70%.

4) A pitfall in evaluation: seeding parallel random number generators by combining a seed with a thread index makes streams that are shifted copies of each other. In our LBR runs this shrank the effective sample size and produced a spurious edge; we give the fix and a test that detects it.
5) An opponent-adaptation layer on the blueprint, built only from information visible to the whole table. It compares each opponent's fold rate per situation with the blueprint's own, values a deviation against both the observed and the blueprint-implied opponent, and deviates only by the difference, within a cap. Against eight opponents it multiplies the win rate by 2.4 to 7.3 against flawed ones and stays within error against sound ones, with one small, measured cost.

Section II reviews related work. Section III defines the game and the information available to an agent. Sections IV and V describe training and evaluation, Section VI the adaptation layer, and Section VII the experiments. Section VIII discusses limitations and Section IX concludes.

II. RELATED WORK

A. EQUILIBRIUM COMPUTATION

Counterfactual regret minimization (CFR) learns an approximate equilibrium of a two-player zero-sum game by self-play, minimizing regret separately at every information set [6]. CFR+ floors regrets at zero and weights later iterations more [7]; discounted CFR generalizes this weighting [8]. Monte Carlo variants sample chance outcomes and opponent actions so that an iteration costs a small part of a full tree walk [9]; Pluribus trained its blueprint with a linearly weighted, externally sampled variant [2]. DeepStack [10], Libratus and Pluribus add search during play, and ReBeL combines search with reinforcement learning [11].

B. ABSTRACTION AND ITS PATHOLOGIES

NLHE is far too large to solve directly, so similar hands are grouped into buckets and only a few bet sizes are allowed. Hands that differ only by suit names are strategically identical and can be merged without loss [12]. Distribution-aware abstractions group hands by their distribution of future strength rather than its mean [5]. Solving a finer abstraction does not always reduce exploitability in the real game [4], and a strategy can grow more exploitable in the real game while it converges in its abstraction [5]. We observe the latter and locate it by betting round.

C. MEASURING STRENGTH

Exact exploitability can be computed only for small games. Local best response [3] gives a lower bound for NLHE: an opponent that knows its own cards, tracks the range of hands the strategy may hold, and picks the action with the best value under a check-down assumption. It showed that several strong competition agents were highly exploitable.

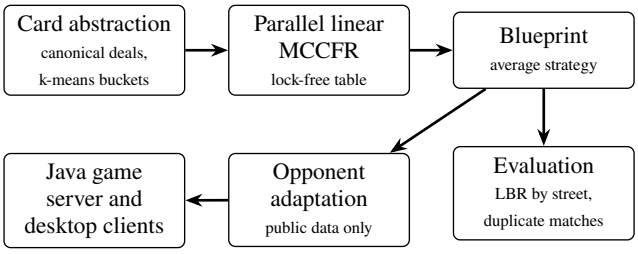


FIGURE 1. The system. A Rust trainer builds the card abstraction and the blueprint; the evaluation tools measure it; the adaptation layer and the Java server bring it to the table.

D. OPPONENT MODELING AND SAFE EXPLOITATION

Early poker agents adjusted to observed action frequencies [13], [14]; Bayesian approaches update a distribution over opponent strategies [15]. Exploitation carries risk, since a fitted strategy can itself be exploited. Restricted Nash response bounds that risk by mixing a model with an equilibrium [16], and safe exploitation bounds the loss relative to the game value [17], [18]. Our adaptation layer uses engineering bounds instead of a formal guarantee; Section VIII discusses the difference.

III. PROBLEM SETTING

We study heads-up NLHE with blinds of 50 and 100 chips and stacks of 100 big blinds (10 000 chips), reset at the start of every hand. Results are in big blinds per 100 hands (bb/100) or in milli-big-blinds per hand (mbb/hand; $1\ bb/100 = 10\ mbb/hand$).

Information boundary. An agent receives exactly what a human player's client receives: its own hole cards, every public action with its amount, the community cards, and the cards of any hand turned over at showdown. In the game server this filtering happens in one routing class; the evaluation harness contains a copy of that router, and a test checks that the two produce identical messages over 300 random hands.

Goal. Train a strategy that is hard to exploit, find where it is weak, and adapt it to opponents whose strategy is unknown and may change, without losing to opponents who change style to punish the adaptation.

IV. BLUEPRINT TRAINING

Fig. 1 shows the system. This section describes the trainer.

A. BETTING ABSTRACTION

The first bet or raise of a betting round may be half the pot, the pot or twice the pot, where "pot" means the pot after calling; the next two raises are pot-sized; moving all-in is always allowed; after three raises a player may only fold, call or move all-in. A player may fold only when facing a bet. An information set consists of the full betting history and the bucket of the player's cards on the current round. Earlier buckets are forgotten (imperfect recall), as in Pluribus.

B. CARD ABSTRACTION

Preflop. The 169 strategically distinct starting hands, which loses nothing.

Canonical deals. A deal is renamed to the smallest of its 24 suit permutations, giving 1 286 792 distinct flops and 13 960 050 distinct turns, the published counts [12].

Flop and turn. For every canonical deal we compute a histogram of the hand's river strength over the cards still to come: on the turn every river card is tried, on the flop 96 random runouts; on each resulting board the strength is estimated against 64 random opponent hands. Histograms have 16 bins and are compared by the squared distance between their cumulative forms, which approximates earth mover's distance. k-means, started by k-means++ and run on all cores, groups them into 200 buckets per round. Buckets are numbered from weakest to strongest mean strength.

River. The exact probability of beating a random hand, over all 990 opponent hands, in 200 buckets of equal size. Both players' river strengths are computed from one set of 1 081 hand scores per board.

Building the abstraction takes about four minutes on twelve cores; the tables occupy 137 MB.

C. PARALLEL LINEAR MCCFR

Each iteration samples one deal, then walks the tree once per player with external sampling [9]: the updated player tries every action, the opponent samples one from its current strategy, and regret matching gives the current strategy at every information set. All threads share one open-addressing hash table of information sets and update it without locks; occasional lost updates add little to the sampling noise [19]. Linear weighting is applied as in Pluribus: training runs in epochs of 10^5 iterations and after epoch e every regret and strategy sum is multiplied by $e/(e+1)$.

Regrets and strategy sums are stored as 64-bit floats. With 32-bit floats the sums at the busiest preflop nodes reached tens of millions after a few million iterations, where an update of about one is lost to rounding and training quietly stalls.

D. RIVER RE-SOLVING

When the river card is dealt, the agent stops playing its river buckets and solves the rest of the hand. The subgame starts at the river with the actual pot and uses the blueprint's bet sizes. Each player's range is the product, over that player's earlier actions, of the probability that the blueprint takes the action with each pair of hole cards; the ranges use no private cards. The opponent's probabilities are floored at 0.01 per action, so that a range never collapses after an action the blueprint rarely takes. This is unsafe re-solving [1]: it trusts the opponent's range.

The subgame is solved by discounted CFR [8] ($\alpha = 1.5$, $\beta = 0$, $\gamma = 2$) in vector form: every decision keeps regrets for each of the up to 1 081 hands that miss the board, and both players are updated in turn. A fold is valued for all hands at once from the total opponent weight minus the weight on hands that share a card. A showdown is valued by sweeping the hands in order of strength while keeping running totals per card, so that card removal is exact and the cost is linear in the number of hands. Hands with negligible weight in both ranges are left out. On a river with 117 subgame nodes and full ranges, 200 iterations take about 0.3 s on one core and leave an exploitability of 0.025 big blinds.

TABLE 1. Training Speed on an Apple M3 Pro (12 Cores), Heads-Up NLHE, 50 Buckets per Round

Change	1 core (iterations per second)	12 cores
First implementation	4 100	–
Fixed-size states and action lists; shared table	5 430	46 517
One deal shared by both players' walks	10 000	84 021
River strengths of both players from one scoring	23 265	199 067

Table 1 shows how speed developed. Sixty million iterations of a 200-bucket abstraction, about 3.08 million information sets, take seven minutes.

V. EVALUATION METHODOLOGY

A. DUPLICATE MATCHES

Two strategies play every deal twice with seats exchanged, so neither profits from better cards. We report the mean and a 95% interval of ± 1.96 standard errors of the per-deal result.

B. LOCAL BEST RESPONSE BY BETTING ROUND

Our LBR follows [3]. It keeps a weight for each of the 1 326 hands the strategy may hold and, after every action of the strategy, multiplies each weight by the probability that the strategy takes that action with that hand. At its own decision it values folding at zero, checking or calling an amount c into a pot P at $W(P+c)-c$, where W is its probability of winning against the weighted range, and a bet that adds d chips at

$$FP + (1-F)\big(W'(P+2d-c)-d\big), \tag{1}$$

where F is the probability that the strategy folds and W' the winning probability against the hands that do not fold. LBR uses the strategy's own bet sizes. Winning probabilities are exact on the river and the turn and sampled on earlier rounds.

LBR's check-down valuation plays the preflop badly: allowed to deviate there, it loses money and hides later leaks. By default it therefore plays the strategy itself before the flop. To locate leaks we also let it deviate on *one* round only and play the strategy everywhere else; since the strategy against itself breaks even, what LBR then wins is what that round alone gives away.

C. A PITFALL IN SEEDING PARALLEL GENERATORS

A first version seeded each thread's generator with `seed XOR (index × step)`, where `step` is the constant that SplitMix64 [20] adds to its state on every draw. For small seeds the resulting states differ by small multiples of `step`, so some threads' streams were exact copies of others shifted by a few draws, and they dealt nearly the same hands. The number of independent hands was much smaller than reported, margins were understated, and an LBR that never

deviates appeared to win $17.7 \pm 4.6\ bb/100$ across eight seeds. Hashing seed and index together before seeding removes the overlap; the same check then gives values scattered around zero (mean -2.4, each ± 36). We keep it as a test, together with a test that neighbouring streams never repeat each other's output. Results in this article use the corrected generator.

VI. OPPONENT ADAPTATION

The adaptation layer sits on top of the blueprint with river re-solving (the Expert agent). It changes play only after the flop, only from information the whole table saw, and only when the evidence says that leaving the blueprint pays. With no evidence it plays exactly as the blueprint.

A. OPPONENT BOOK

For each opponent the agent counts, over all observed hands, voluntary entries, preflop raises, postflop aggression and, from cards shown at showdown, how often postflop betting was a bluff. Folding is recorded *by situation*: whether the opponent faced the first bet of a betting round or a raise, and the price of the bet, $c/(P+c)$ for a call of c into a pot P, in three classes (below 0.25, 0.25 to 0.4, above 0.4). This gives six cells. A single fold rate was not enough: sound opponents fold often to large bets and rarely to small ones, and a global rate made them look exploitable when they were not.

Each cell's fold rate is compared with the rate at which the blueprint itself folds in that cell, measured once in self-play (0.40, 0.44, 0.74 after a first bet of increasing price; 0.13, 0.60, 0.60 after a raise). A cell seen n times contributes the shift

$$\delta = \frac{n}{n+20}\left(\hat{f} - f_{\mathrm{bp}}\right), \tag{2}$$

set to zero when $|\delta| < 0.1$. This dead zone keeps sampling noise against sound opponents from being read as a weakness.

B. TWO VALUATIONS

At each postflop decision the agent values its actions as local best response does (Section V): against a range for the opponent, with the opponent's reply to a bet given by a fold probability, and checking down afterwards. It does this twice, with the same sampled cards (common random numbers) so that the difference is not noise:

- *believed*: the opponent's range implied by the blueprint for the actions so far, and the blueprint's own fold probabilities;
- *observed*: the same range tempered by the opponent's measured aggression and bluffing (widened when it bets more and bluffs more than the blueprint does, baselines 0.27 and 0.55), and fold probabilities shifted by (2). Extra folds remove the weakest hands of the range first, as a real player's folds would.

Let a^* be the best action under the observed valuation, V^o, and $\bar{V}$ the value of the blueprint's own mixed strategy. The gain of deviating is what a^* adds under the observed model beyond what it would add under the believed one:

$$g = \left(V^o(a^*) - \bar{V}^o\right) - \max\left(0,\ V^b(a^*) - \bar{V}^b\right). \tag{3}$$

TABLE 2. Checks Against Published Values

Quantity	Ours	Published
Five-card hands per category	match	standard
Seven-card hands per category	match	standard
Canonical flops	1 286 792	1 286 792 [12]
Canonical turns	13 960 050	13 960 050 [12]
Kuhn, uniform play: exploitability	0.458333	0.458333
Leduc, uniform play: exploitability	2.373611	2.373611
Kuhn: first player's value	-0.05556	$-1/18$
Leduc: first player's value	-0.0856	-0.0856

Hand counts: all 2 598 960 five-card and 133 784 560 seven-card hands. Uniform-play values are half of OpenSpiel's NashConv.

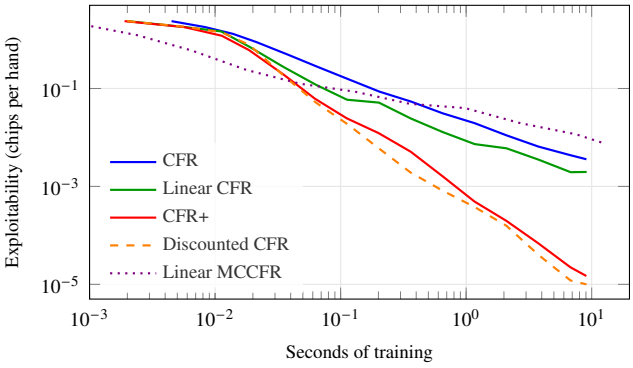


FIGURE 2. Exploitability of each solver's average strategy on Leduc Hold'em against training time on one core. Exploitability is computed exactly by best response.

Subtracting the believed term removes actions that look good only because the valuation itself is crude (the check-down model favors some actions against any opponent). The agent plays a^* with probability

$$\lambda = 0.6 \cdot \frac{h}{h+30} \cdot \min\left(1,\ \frac{g}{0.25\,P}\right) \tag{4}$$

for h hands observed and pot P, and the blueprint's strategy otherwise. The cap of 0.6 keeps some blueprint play in every spot, so a wrong or manipulated read cannot remove it entirely.

An earlier version of the layer, on a simulation-based agent, raised its win rate against all six opponent types tested; we report the blueprint version here.

VII. EXPERIMENTS

A. SOLVER VALIDATION

Table 2 lists the checks against published values, and Fig. 2 shows convergence on Leduc Hold'em. On this small game, walking the whole tree with CFR+ or discounted CFR is fastest; sampling pays off only in games too large to walk.

B. WHERE THE BLUEPRINT LEAKS

Table 3 and Fig. 3 give LBR results for blueprints trained on the 200-bucket abstraction for two, twenty and sixty million iterations. Each value uses 100 000 deals (200 000 hands).

The flop is close to unexploitable at every length. The turn leaks moderately. The river leaks most, and more the longer

TABLE 3. LBR Winnings (mbb/hand) Against the Blueprint, by the Round on Which LBR May Deviate

Iterations	Flop	Turn	River	Postflop	Margin
2×10^6	-109	91	276	479	± 75
2×10^7	-10	220	586	635	± 75
6×10^7	-9	190	736	793	± 75

"Postflop" lets LBR deviate on the flop, turn and river. Margins are the largest 95% interval in each row.

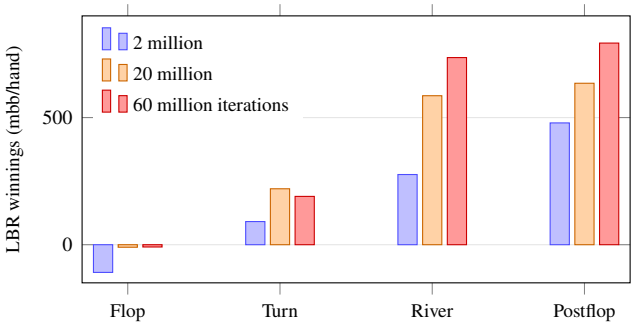


FIGURE 3. What each betting round gives away. LBR deviates only on the round named (or on all three after the flop) and plays the blueprint elsewhere. The river leak grows with training.

the strategy trains. Yet training is not failing: in the same abstraction family, a 20-million-iteration strategy beats a 2-million one head to head by 18.5 ± 3.6 and 13.5 ± 3.6 $bb/100$ (two seeds) and a 200 000 one by 68.1 ± 5.0 $bb/100$, over 400 000 hands each[1]. Training sharpens play within the river buckets, and an opponent with exact cards punishes the distinctions those buckets cannot draw: a river bucket measures strength against a random hand, which says little about strength against the narrow range an opponent holds after three rounds of betting. This matches the overfitting described in [5] and locates it.

C. FIXING THE RIVER

Richer buckets were tried first. With opponent cluster hand strength [5], a river hand is described by its probability of beating each of eight clusters of starting hands, and k-means on these vectors gives 200 buckets (flop and turn unchanged). River-only LBR was 359, 635 and 695 mbb/hand at the three training lengths, no different from Table 3 within the margins, and still rising with training. The leak does not come from what a bucket knows, but from bucketing itself: every hand in a bucket must play alike, and an opponent with exact cards reads the lines training draws between buckets.

Re-solving removes it (Table 4). LBR that may deviate only on the river wins nothing measurable, and exploitability after the flop falls by 70% for the longer-trained blueprint. With re-solving, longer training no longer hurts. The remaining exploitability is mostly on the turn.

[1]These matches used the earlier 50-bucket abstraction.

TABLE 4. LBR Winnings (mbb/hand) With River Buckets and With River Re-solving

	River only		Postflop	
Iterations	Buckets	Re-solved	Buckets	Re-solved
2×10^6	276 ± 67	-59 ± 90	479 ± 75	275 ± 103
6×10^7	736 ± 57	7 ± 64	793 ± 75	243 ± 97

200 histogram buckets; re-solving with 200 iterations; 100 000 hands per value.

TABLE 5. Heads-Up Win Rates (bb/100, 95% Intervals) of the Expert Agent With and Without Adaptation; 20 000 Hands Unless Noted

Opponent	Blueprint	Adaptive
Caller	243 ± 31	1770 ± 75
Maniac	273 ± 45	643 ± 59
Chameleon	88 ± 16	367 ± 33
Over-folder	45 ± 9	38 ± 9
Solid	37 ± 10	34 ± 11
Easy	27 ± 12	22 ± 13
Simulation	16 ± 14	17 ± 17
Hard (60 000 hands)	34 ± 10	17 ± 12

D. OPPONENT ADAPTATION

Table 5 reports heads-up matches of the Expert agent (the 60-million-iteration blueprint with river re-solving) with and without the adaptation layer, against eight opponents, all with the same seed. Three are sound agents: Easy and Solid are rule-based; Simulation estimates its equity by dealing out the unseen cards; Hard is Simulation with a bounded opponent model of its own. Four have deliberate flaws: Caller never folds; Maniac bets or raises 65% of the time; Over-folder folds to any postflop bet without a strong hand; Chameleon alternates between over-folding and calling every 1 500 decisions, to punish an agent that adapts.

Against the three opponents with large flaws the layer multiplies the win rate: by 7.3 against Caller, 2.4 against Maniac and 4.2 against Chameleon, which switches style to punish adaptation. Against the five others the two agents are within each other's intervals, except Hard, where the adaptive agent gives up about 17 bb/100: Hard's own adjustments move it away from the blueprint in ways our fold model reads, wrongly, as weakness. Both versions beat every opponent.

The Over-folder result is worth noting. It folds too often, but the blueprint already exploits that by betting often; its fold rate per situation is close enough to the blueprint's own that the dead zone holds the layer back. Earlier versions without situational fold rates shoved all-in against it and lost.

Table 6 shows the trade-off between exploitation and safety as the cap and the dead zone change. A cap of 0.15 already earns half the gain against Caller; raising it to 0.6 without the dead zone gains a little more against weak opponents and loses against the sound ones (Simulation -33, Hard -4). Switching off single parts of the layer (tempering, fold shifts, only aggressive or only passive deviations) changed results by less than their margins at 20 000 hands, so we do not report those ablations separately.

TABLE 6. Exploitation Against Safety: Win Rates (bb/100, 20 000 Hands) for Settings of the Cap and the Dead Zone; Intervals ±10 to ±75

Setting	Caller	Maniac	Chameleon	Simulation	Hard
Blueprint	243	273	88	16	32
Cap 0.15	814	403	190	9	14
Cap 0.3	1261	526	275	−2	2
Cap 0.6, no dead zone	1832	703	412	−33	−4
Cap 0.6, dead zone 0.1	1770	643	378	17	6

VIII. LIMITATIONS

Unsafe re-solving. The river solver trusts the opponent's range as the blueprint implies it. An opponent who plays differently before the river can make that range wrong; safe re-solving [1] would bound the damage. LBR after the flop does deviate before the river, and the result still improved, but this is not a guarantee.

LBR is a lower bound. It uses the strategy's own bet sizes and a check-down valuation; a full best response would win more.

Abstraction. The river leak shows the limits of strength-against-random buckets; all results are for one betting abstraction.

Synthetic opponents. All opponents in Table 5 were written by us, and four are extreme by design. Gains against people will be smaller. Chameleon tests one switching pattern, not an adversary that models our agent, and against Hard, which does adapt, the layer costs about 17 bb/100.

Informal safety. The cap and the dead zone bound how often the adaptive agent leaves the blueprint, not its exploitability; a restricted Nash response [16] on top of the blueprint would give a formal bound.

No external benchmark or human study yet. [Matches against Slumbot; a study with human players.]

IX. CONCLUSION

A consumer laptop trains a heads-up NLHE blueprint at about 2×10^5 iterations per second. Restricting local best response to one betting round turns a single exploitability number into a map of where a strategy is weak; for our strategies the map points at the river, whose leak grows as the strategy overfits its card abstraction. Richer buckets do not remove it; solving the river hand by hand during play does, and cuts exploitability after the flop by 70%. Careful seeding of parallel generators matters as much as the solver for trustworthy measurements. A bounded opponent model built from public information, which leaves the blueprint only when the evidence beats the blueprint's own behavior, multiplies win rates against flawed opponents and costs little against sound ones. Next steps are re-solving from the turn, a restricted Nash response for a formal safety bound, and external benchmarks against Slumbot and human players.

DATA AND CODE AVAILABILITY

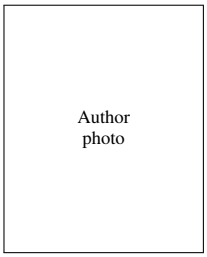
The game engine, server, solvers and evaluation tools, with the commands and seeds behind every result, are available at [https://github.com/SakD2006/holdem (to be made public, with a fixed release)].

ACKNOWLEDGMENT

The author used Claude (Anthropic), an AI assistant, to help write and review the software described here and to draft and edit the text of all sections of this article. The author designed the study, ran the experiments, checked every result and reference, and takes responsibility for the content. [Adjust this statement to match actual use; add anyone who helped but is not an author.]

REFERENCES

[1] N. Brown and T. Sandholm, "Superhuman AI for heads-up no-limit poker: Libratus beats top professionals," *Science*, vol. 359, no. 6374, pp. 418–424, 2018.
[2] N. Brown and T. Sandholm, "Superhuman AI for multiplayer poker," *Science*, vol. 365, no. 6456, pp. 885–890, 2019.
[3] V. Lisý and M. Bowling, "Equilibrium approximation quality of current no-limit poker bots," in *Proc. AAAI Workshop Comput. Poker Imperfect Inf. Games*, 2017.
[4] K. Waugh, D. Schnizlein, M. Bowling, and D. Szafron, "Abstraction pathologies in extensive games," in *Proc. 8th Int. Conf. Auton. Agents Multiagent Syst. (AAMAS)*, 2009, pp. 781–788.
[5] M. Johanson, N. Burch, R. Valenzano, and M. Bowling, "Evaluating state-space abstractions in extensive-form games," in *Proc. 12th Int. Conf. Auton. Agents Multiagent Syst. (AAMAS)*, 2013, pp. 271–278.
[6] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione, "Regret minimization in games with incomplete information," in *Adv. Neural Inf. Process. Syst.*, vol. 20, 2008, pp. 1729–1736.
[7] O. Tammelin, "Solving large imperfect information games using CFR+," 2014, *arXiv:1407.5042*.
[8] N. Brown and T. Sandholm, "Solving imperfect-information games via discounted regret minimization," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 1829–1836.
[9] M. Lanctot, K. Waugh, M. Zinkevich, and M. Bowling, "Monte Carlo sampling for regret minimization in extensive games," in *Adv. Neural Inf. Process. Syst.*, vol. 22, 2009, pp. 1078–1086.
[10] M. Moravčík *et al.*, "DeepStack: Expert-level artificial intelligence in heads-up no-limit poker," *Science*, vol. 356, no. 6337, pp. 508–513, 2017.
[11] N. Brown, A. Bakhtin, A. Lerer, and Q. Gong, "Combining deep reinforcement learning and search for imperfect-information games," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 17057–17069.
[12] K. Waugh, "A fast and optimal hand isomorphism algorithm," in *Proc. AAAI Workshop Comput. Poker Imperfect Inf.*, 2013.
[13] D. Billings, D. Papp, J. Schaeffer, and D. Szafron, "Opponent modeling in poker," in *Proc. 15th Nat. Conf. Artif. Intell. (AAAI)*, 1998, pp. 493–499.
[14] D. Billings, A. Davidson, J. Schaeffer, and D. Szafron, "The challenge of poker," *Artif. Intell.*, vol. 134, nos. 1–2, pp. 201–240, 2002.
[15] F. Southey *et al.*, "Bayes' bluff: Opponent modelling in poker," in *Proc. 21st Conf. Uncertainty Artif. Intell. (UAI)*, 2005, pp. 550–558.
[16] M. Johanson, M. Zinkevich, and M. Bowling, "Computing robust counter-strategies," in *Adv. Neural Inf. Process. Syst.*, vol. 20, 2008, pp. 721–728.
[17] S. Ganzfried and T. Sandholm, "Game theory-based opponent modeling in large imperfect-information games," in *Proc. 10th Int. Conf. Auton. Agents Multiagent Syst. (AAMAS)*, 2011, pp. 533–540.
[18] S. Ganzfried and T. Sandholm, "Safe opponent exploitation," *ACM Trans. Econ. Comput.*, vol. 3, no. 2, Art. no. 8, 2015.
[19] B. Recht, C. Ré, S. Wright, and F. Niu, "Hogwild!: A lock-free approach to parallelizing stochastic gradient descent," in *Adv. Neural Inf. Process. Syst.*, vol. 24, 2011, pp. 693–701.
[20] G. L. Steele, Jr., D. Lea, and C. H. Flood, "Fast splittable pseudorandom number generators," in *Proc. ACM Int. Conf. Object Oriented Program. Syst. Lang. Appl. (OOPSLA)*, 2014, pp. 453–472.



SAKSHAM DUBEY [is pursuing the B.Tech. degree in [branch] at the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India, since 2024. His research interests include game-playing artificial intelligence, imperfect-information games and systems programming.]